

PRINCIPLES OF PROGRAMMING LANGUAGES / LESSON GUIDE

06 Input and validation

Modules 1-3: representations, contracts, and boundary functions

1. Introduction & core concept

Input validation establishes whether external data satisfies a program's declared contract before the program relies on it. Reading, parsing, and validation are distinct stages: reading obtains bytes or text; parsing interprets a representation; validation checks domain rules. A parseable number may still be outside the allowed range, and a successful prefix parse may leave unwanted text unread.

The acceptance policy belongs to the application. Decide whether signs, surrounding whitespace, leading zeros, non-ASCII digits, empty input, and trailing characters are permitted. Validation is not merely 'catch any exception': expected rejection should have a clear explanation, and unrelated defects should not be silently converted into bad-input messages.

2. Under the hood: the mechanics

A boundary function converts an external representation into a trustworthy internal value. A useful pipeline is decode, normalize according to policy, recognize the complete lexical form, convert, then check semantic constraints. Whole-input recognition prevents a numeric prefix such as 7 in '7cats' from being treated as the entire input.

Runtime APIs have different grammars. Python `int` converts integer text; JavaScript `Number` accepts forms an application may wish to reject; C++ `from_chars` reports both a status and where parsing stopped [1-3]. A program using finite standard input also reaches EOF: it does not have an unlimited interactive conversation unless that behavior is explicitly implemented.

Learning outcomes

- Specify a complete input contract.
- Separate lexical rejection from range rejection.
- Check complete consumption and recover without using an invalid value.

Examples progress from Python 3 to modern JavaScript to C++17. Code follows each language's native syntax. Bracketed citations refer to the final page.

3. CODE EXAMPLES / PYTHON

Simple: validate a bounded integer

```
import re

def parse_count(text):
    text = text.strip()
    if re.fullmatch(r"[0-9]+", text) is None:
        raise ValueError("Use decimal digits only")
    value = int(text)
    if not 0 <= value <= 10:
        raise ValueError("Use a value from 0 to 10")
    return value

for text in [" 7 ", "3.5", "11"]:
    try:
        print(parse_count(text) ** 2)
    except ValueError as error:
        print(error)
```

In-depth reasoning

Expected result: 49

Use decimal digits only

Use a value from 0 to 10

The policy permits surrounding whitespace and one or more ASCII digits. `strip` normalizes only the allowed surrounding whitespace. `fullmatch` requires the entire remaining string to satisfy the grammar, so decimals, signs, empty strings, and trailing letters are rejected. Leading zeros are accepted by this policy.

For ' 7 ', normalization produces '7', conversion yields 7, the range check succeeds, and squaring produces 49. '3.5' fails lexical recognition before conversion. '11' parses successfully but fails the domain constraint. Both failures use `ValueError`, but their messages explain different violated conditions.

The function's successful return guarantees an integer from zero through ten. The caller can then square it without repeating those checks. For an exposed service, bound input length as well: a huge string can cost memory or conversion time even when it will eventually be rejected. This classroom function demonstrates semantics rather than a complete service boundary.

Compare the language models

Python `int` alone is not the chosen grammar. JavaScript `Number` and C++ `from_chars` also need surrounding policy checks. Equivalent contracts require deliberately aligning whitespace, signs, accepted digits, and full-consumption rules.

Checkpoint: Is '+7' accepted? No. That is a contract choice; the numeric value being meaningful does not override the grammar.

3. CODE EXAMPLES / JAVASCRIPT

Intermediate: reject partial and empty conversions

```
function parseCount(text) {
  const trimmed = text.trim();
  if (!/^[0-9]+$/.test(trimmed)) {
    throw new TypeError("Use decimal digits only");
  }
  const value = Number(trimmed);
  if (!Number.isSafeInteger(value) || value > 10) {
    throw new RangeError("Use a value from 0 to 10");
  }
  return value;
}

for (const text of ["7", "", "7cats"]) {
  try {
    console.log(parseCount(text));
  } catch (error) {
    console.log(error.name);
  }
}
```

In-depth reasoning

Expected result: 7

TypeError

TypeError

The function assumes its argument is a string. `trim` creates the normalized text, and the regular expression rejects empty content or any non-digit content. The end anchor is safe for this policy because surrounding whitespace, including final line breaks, was already trimmed. The accepted digits can only express a nonnegative value.

Number conversion occurs only after lexical recognition. `Number.isSafeInteger` rejects non-integer, non-finite, or unsafe integer results, and the application bound rejects values above ten. Empty text never reaches `Number`, avoiding its otherwise surprising conversion to zero. `'7cats'` is rejected rather than partially parsed as seven.

The loop shows that invalid entries do not prevent later entries from being examined. Catching error broadly is appropriate here only because the example intentionally reports names from a small local computation. Production boundaries should distinguish expected validation errors from unrelated programmer mistakes and preserve diagnostic information.

Compare the language models

Python can use `ValueError` for both lexical and range errors. This JavaScript interface distinguishes `TypeError` and `RangeError` by policy. C++ `from_chars` reports conversion errors without throwing, so callers can encode rejection as a Boolean, optional value, or richer result type.

Checkpoint: Why is `parseInt` alone insufficient for this contract? It can accept a valid numeric prefix while leaving trailing text unvalidated.

3. CODE EXAMPLES / C++

Advanced: inspect status and the unconsumed suffix

```
#include <charconv>
#include <iostream>
#include <string_view>
#include <system_error>

bool parse_count(std::string_view text, int& output) {
    if (text.empty()) return false;
    int parsed = 0;
    const char* first = text.data();
    const char* last = first + text.size();
    auto result = std::from_chars(first, last, parsed);
    if (result.ec != std::errc{} || result.ptr != last ||
        parsed < 0 || parsed > 10) {
        return false;
    }
    output = parsed;
    return true;
}

int main() {
    int value = -1;
    std::cout << std::boolalpha;
    std::cout << parse_count("7", value) << ' ' << value << '\n';
    std::cout << parse_count("7cats", value) << ' ' << value << '\n';
}
```

In-depth reasoning

Expected result: true 7
false 7

`from_chars` parses a character range without skipping leading whitespace, returning an error code and stopping pointer [3]. '7' is accepted; '7cats' leaves an unconsumed suffix; '7' fails conversion. The empty-input guard also avoids forming a character range from a default empty view.

Only a completely parsed, in-range value is assigned to output. The first call commits 7. The second returns false and leaves that previous value unchanged; it is not a new successful parse. C++17 sequences these stream operations so each printed value follows its parse call.

The API permits a leading minus but not a leading plus. The range check rejects negatives, yet '-0' is accepted. This differs from the earlier digits-only grammar and whitespace policy. Aligning contracts requires explicit normalization and recognition; choosing a numeric conversion API alone is insufficient. The sample views refer to live string literals.

Compare the language models

Python/JavaScript return a value or raise. This C++ interface returns a success flag and writes through a reference only on success. Compare both error transport and accepted text when translating.

Checkpoint: Why check `result.ptr` even when `result.ec` indicates success? Conversion may have consumed only a prefix.

4. CONSOLIDATION / REFERENCES

Reason, compare, and apply

The central lesson

Write the acceptance policy before selecting parsing APIs. In Paradigm Diagnostics, the Input and validation lesson uses the preset input 7 and demonstrates a successful square. Its finite input snapshot is part of the execution report. Do not assume an input box or terminal will supply extra data after EOF.

Review questions with answer guidance

1. Which tests reveal policy differences?

Empty text, leading/trailing whitespace, '+7', '-0', '007', non-ASCII digits, huge integers, and '7cats'.

2. Why delay committing output?

A failed parse should not leave a caller using a partially initialized or unvalidated value.

A short learning activity

Predict each example's outcome before running it. Change one input or one operation, state the rule you expect to apply, and compare the observation with the prediction. Explain the earliest point where behavior changes. A matching output supports that particular execution; it does not prove general program equivalence.

References and further reading

[1] Python: int conversion. [Read source](#)

[2] ECMAScript: numeric conversion. [Read source](#)

[3] C++ working draft: from_chars. [Read source](#)

References are official language/runtime documentation or the public C++ working draft. Links checked 30 September 2026. Examples and reasoning are original teaching material; implementation details and scheduling outcomes are identified where relevant.